

Figure 8.1 Linear filtering of random processes

where $H(\omega)$ is the frequency response of the filter. Therefore the value of the output PDS at frequency ω depends on the squared magnitude response of the filter and the input PDS at the same frequency.

Another important relationships between $x(n)$ and $y(n)$ are

$$m_y = E\left[\sum_{l=-\infty}^{\infty} h(l)x(n-l)\right] = \sum_{l=-\infty}^{\infty} h(l)E[x(n-l)] = m_x \sum_{l=-\infty}^{\infty} h(l), \tag{8.1.30}$$

and

$$r_{yx}(k) = E[y(n+k)x(n)] = E\left[\sum_{l=-\infty}^{\infty} h(l)x(n+k-l)x(n)\right]$$

$$= \sum_{l=-\infty}^{\infty} h(l)r_{xx}(k-l) = h(k) * r_{xx}(k). \tag{8.1.31}$$

Taking the z-transform of both sides, we obtain

$$P_{yx}(z) = H(z)P_{xx}(z). \tag{8.1.32}$$

Similarly, the relationships between the input and the output signals are

$$r_{xy}(k) = \sum_{l=-\infty}^{\infty} h(l)r_{xx}(k+l) = h(k) * r_{xx}(-k) \tag{8.1.33}$$

and

$$P_{yx}(z) = H^*(z)P_{xx}(z). \tag{8.1.34}$$

If the input signal $x(n)$ is a zero-mean white noise with the autocorrelation function defined in (8.1.26), Equation (8.1.31) becomes

$$r_{yx}(k) = \sum_{l=-\infty}^{\infty} h(l)\sigma_x^2\delta(k-l) = \sigma_x^2 h(k). \tag{8.1.35}$$

This equation shows that by computing the crosscorrelation function $r_{yx}(k)$, the impulse response $h(n)$ of a filter (or system) can be obtained. This fact can be used to estimate an unknown system such as the room impulse response used in Chapter 4.

Example 8.4: Let the system shown in Figure 8.1 be a second-order FIR filter. The input $x(n)$ is a zero-mean white noise given by Example 8.3, and the I/O equation is expressed as

$$y(n) = x(n) + 3x(n-1) + 2x(n-2).$$

Find the mean m_y and the autocorrelation function $r_{yy}(k)$ of the output $y(n)$.

(a) $m_y = E[y(n)] = E[x(n)] + 3E[x(n-1)] + 2E[x(n-2)] = 0.$

(b) $r_{yy}(k) = E[y(n+k)y(n)]$

$$= 14r_{xx}(k) + 9r_{xx}(k-1) + 9r_{xx}(k+1) + 2r_{xx}(k-2) + 2r_{xx}(k+2)$$

$$= \begin{cases} 14\sigma_x^2 & \text{if } k = 0 \\ 9\sigma_x^2 & \text{if } k = \pm 1 \\ 2\sigma_x^2 & \text{if } k = \pm 2 \\ 0 & \text{otherwise.} \end{cases}$$

8.2 Adaptive Filters

Many practical applications involve the reduction of noise and distortion for extraction of information from the received signal. The signal degradation in some physical systems is time varying, unknown, or possibly both. Adaptive filters provide a useful approach for these applications. Adaptive filters modify their characteristics to achieve certain objectives and usually accomplish the modification (adaptation) automatically. For example, consider a high-speed modem for transmitting and receiving data over telephone channels. It employs a filter called a channel equalizer to compensate for the channel distortion. Since the dial-up communication channels have different characteristics on each connection and are time varying, the channel equalizers must be adaptive.

Adaptive filters have received considerable attention from many researchers over the past 30 years. Many adaptive filter structures and adaptation algorithms have been developed for different applications. This chapter presents the most widely used adaptive filter based on the FIR filter with the LMS algorithm. Adaptive filters in this class are relatively simple to design and implement. They are well understood with regard to convergence speed, steady-state performance, and finite-precision effects.

8.2.1 Introduction to Adaptive Filtering

An adaptive filter consists of two distinct parts – a digital filter to perform the desired signal processing, and an adaptive algorithm to adjust the coefficients (or weights) of that filter. A general form of adaptive filter is illustrated in Figure 8.2, where $d(n)$ is a desired signal (or primary input signal), $y(n)$ is the output of a digital filter driven by a reference input signal $x(n)$, and an error signal $e(n)$ is the difference between $d(n)$ and $y(n)$. The function of the adaptive algorithm is to adjust the digital filter coefficients to

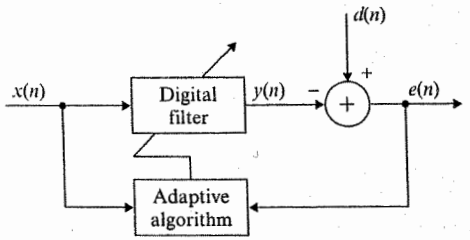


Figure 8.2 Block diagram of adaptive filter

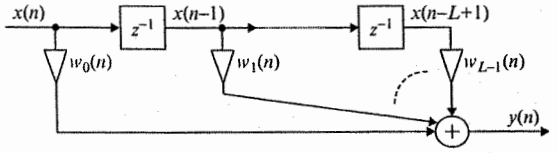


Figure 8.3 Block diagram of FIR filter for adaptive filtering

minimize the mean-square value of $e(n)$. Therefore the filter weights are updated so that the error is progressively minimized on a sample-by-sample basis.

In general, there are two types of digital filters that can be used for adaptive filtering: FIR and IIR filters. The choice of an FIR or an IIR filter is determined by practical considerations. The FIR filter is always stable and can provide a linear phase response. On the other hand, the IIR filter involves both zeros and poles. Unless they are properly controlled, the poles in the filter may move outside the unit circle and make the filter unstable. Because the filter is required to be adaptive, the stability problems are much difficult to handle. Thus the FIR adaptive filter is widely used for real-time applications. The discussions in the following sections will be restricted to the class of adaptive FIR filters.

The most widely used adaptive FIR filter is depicted in Figure 8.3. Given a set of L coefficients, $w_l(n)$, $l = 0, 1, \ldots, L-1$, and a data sequence, $\{x(n)\ x(n-1)\ \ldots x(n-L+1)\}$, the filter output signal is computed as

$$y(n) = \sum_{l=0}^{L-1} w_l(n)x(n-l), \tag{8.2.1}$$

where the filter coefficients $w_l(n)$ are time varying and updated by the adaptive algorithms that will be discussed next.

We define the input vector at time n as

$$\mathbf{x}(n) \equiv [x(n)\ x(n-1)\ldots x(n-L+1)]^T \tag{8.2.2}$$

and the weight vector at time n as

$$\mathbf{w}(n) \equiv [w_0(n)\ w_1(n)\ldots w_{L-1}(n)]^T. \tag{8.2.3}$$

Then the output signal $y(n)$ in (8.2.1) can be expressed using the vector operation

$$y(n) = \mathbf{w}^T(n)\mathbf{x}(n) = \mathbf{x}^T(n)\mathbf{w}(n). \tag{8.2.4}$$

The filter output $y(n)$ is compared with the desired response $d(n)$, which results in the error signal

$$e(n) = d(n) - y(n) = d(n) - \mathbf{w}^T(n)\mathbf{x}(n). \tag{8.2.5}$$

In the following sections, we assume that $d(n)$ and $x(n)$ are stationary, and our objective is to determine the weight vector so that the performance (or cost) function is minimized.

8.2.2 Performance Function

The general block diagram of the adaptive filter shown in Figure 8.2 updates the coefficients of the digital filter to optimize some predetermined performance criterion. The most commonly used performance measurement is based on the mean-square error (MSE) defined as

$$\xi(n) \equiv E[e^2(n)]. \tag{8.2.6}$$

For an adaptive FIR filter, $\xi(n)$ will depend on the L filter weights $w_0(n)$, $w_1(n)$, $\ldots, w_{L-1}(n)$. The MSE function can be determined by substituting (8.2.5) into (8.2.6), expressed as

$$\xi(n) = E[d^2(n)] - 2\mathbf{p}^T\mathbf{w}(n) + \mathbf{w}^T(n)\mathbf{R}\mathbf{w}(n), \tag{8.2.7}$$

where $\mathbf{p}$ is the crosscorrelation vector defined as

$$\mathbf{p} \equiv E[d(n)\mathbf{x}(n)] = [r_{dx}(0)\ r_{dx}(1)\ldots r_{dx}(L-1)]^T, \tag{8.2.8}$$

and

$$r_{dx}(k) \equiv E[d(n)x(n-k)] \tag{8.2.9}$$

is the crosscorrelation function between $d(n)$ and $x(n)$. In (8.2.7), $\mathbf{R}$ is the input autocorrelation matrix defined as

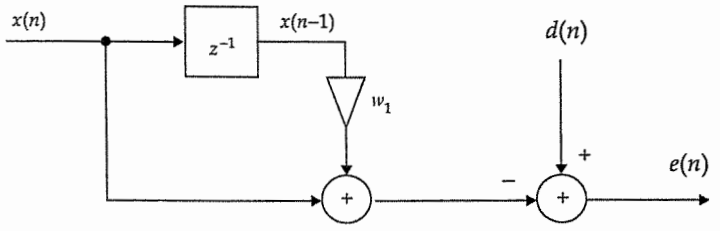
$$\mathbf{R} \equiv E[\mathbf{x}(n)\mathbf{x}^T(n)] = \begin{bmatrix} r_{xx}(0) & r_{xx}(1) & \cdots & r_{xx}(L-1) \\ r_{xx}(1) & r_{xx}(0) & \cdots & r_{xx}(L-2) \\ \vdots & \cdots & \ddots & \vdots \\ r_{xx}(L-1) & r_{xx}(L-2) & \cdots & r_{xx}(0) \end{bmatrix}, \tag{8.2.10}$$

where

$$r_{xx}(k) \equiv E[x(n)x(n-k)] \tag{8.2.11}$$

is the autocorrelation function of $x(n)$.

Example 8.5: Given an optimum filter illustrated in the following figure:



If $E[x^2(n)] = 1$, $E[x(n)x(n-1)] = 0.5$, $E[d^2(n)] = 4$, $E[d(n)x(n)] = -1$, and $E[d(n)x(n-1)] = 1$. Find ξ.

From (8.2.10), $\mathbf{R} = \begin{bmatrix} 1 & 0.5 \\ 0.5 & 1 \end{bmatrix}$, and from (8.2.8), we have $\mathbf{P} = \begin{bmatrix} -1 \\ 1 \end{bmatrix}$.

Therefore from (8.2.7), we obtain

$$\begin{aligned} \xi &= E[d^2(n)] - 2\mathbf{p}^T\mathbf{w} + \mathbf{w}^T\mathbf{R}\mathbf{w} \\ &= 4 - 2[-1 \quad 1]\begin{bmatrix} 1 \\ w_1 \end{bmatrix} + [1 \quad w_1]\begin{bmatrix} 1 & 0.5 \\ 0.5 & 1 \end{bmatrix}\begin{bmatrix} 1 \\ w_1 \end{bmatrix} = w_1^2 - w_1 + 7. \end{aligned}$$

The optimum filter $\mathbf{w}^o$ minimizes the MSE cost function $\xi(n)$. Vector differentiation of (8.2.7) gives $\mathbf{w}^o$ as the solution to

$$\mathbf{R}\mathbf{w}^o = \mathbf{p}. \tag{8.2.12}$$

This system equation defines the optimum filter coefficients in terms of two correlation functions – the autocorrelation function of the filter input and the crosscorrelation function between the filter input and the desired response. Equation (8.2.12) provides a solution to the adaptive filtering problem in principle. However, in many applications, the signal may be non-stationary. This linear algebraic solution, $\mathbf{w}^o = \mathbf{R}^{-1}\mathbf{p}$, requires continuous estimation of $\mathbf{R}$ and $\mathbf{p}$, a considerable amount of computations. In addition, when the dimension of the autocorrelation matrix is large, the calculation of $\mathbf{R}^{-1}$ may present a significant computational burden. Therefore a more useful algorithm is obtained by developing a recursive method for computing $\mathbf{w}^o$, which will be discussed in the next section.

To obtain the minimum MSE, we substitute the optimum weight vector $\mathbf{w}^o = \mathbf{R}^{-1}\mathbf{p}$ for $\mathbf{w}(n)$ in (8.2.7), resulting in

$$\xi_{\min} = E[d^2(n)] - \mathbf{p}^T\mathbf{w}^o. \tag{8.2.13}$$

Since $\mathbf{R}$ is positive semidefinite, the quadratic form on the right-hand side of (8.2.7) indicates that any departure of the weight vector $\mathbf{w}(n)$ from the optimum $\mathbf{w}^o$ would increase the error above its minimum value. In other words, the error surface is concave and possesses a unique minimum. This feature is very useful when we utilize search techniques in seeking the optimum weight vector. In such cases, our objective is to develop an algorithm that can automatically search the error surface to find the optimum weights that minimize $\xi(n)$ using the input signal $x(n)$ and the error signal $e(n)$.

Example 8.6: Consider a second-order FIR filter with two coefficients w_0 and w_1, the desired signal $d(n) = \sqrt{2}\sin(n\omega_0)$, $n \geq 0$, and the reference signal $x(n) = d(n-1)$. Find $\mathbf{w}^o$ and $\xi_{\min}$

Similar to Example 8.2, we can obtain $r_{xx}(0) = E[x^2(n)] = E[d^2(n)] = 1$, $r_{xx}(1) = \cos(\omega_0)$, $r_{xx}(2) = \cos(2\omega_0)$, $r_{dx}(0) = r_{xx}(1)$, and $r_{dx}(1) = r_{xx}(2)$. From (8.2.12), we have

$$\mathbf{w}^o = \mathbf{R}^{-1}\mathbf{p} = \begin{bmatrix} 1 & \cos(\omega_0) \\ \cos(\omega_0) & 1 \end{bmatrix}^{-1}\begin{bmatrix} \cos(\omega_o) \\ \cos(2\omega_0) \end{bmatrix} = \begin{bmatrix} 2\cos(\omega_0) \\ -1 \end{bmatrix}.$$

From (8.2.13), we obtain

$$\xi_{\min} = 1 - [\cos(\omega_0) \quad \cos(2\omega_0)]\begin{bmatrix} 2\cos(\omega_0) \\ -1 \end{bmatrix} = 0.$$

Equation (8.2.7) is the general expression for the performance function of an adaptive FIR filter with given weights. That is, the MSE is a function of the filter coefficient vector $\mathbf{w}(n)$. It is important to note that the MSE is a quadratic function because the weights appear only to the first and second degrees in (8.2.7). For each coefficient vector $\mathbf{w}(n)$, there is a corresponding (scalar) value of MSE. Therefore the MSE values associated with $\mathbf{w}(n)$ form an $(L+1)$-dimensional space, which is commonly called the MSE surface, or the performance surface.

For $L = 2$, this corresponds to an error surface in a three-dimensional space. The height of $\xi(n)$ corresponds to the power of the error signal $e(n)$ that results from filtering the signal $x(n)$ with the coefficients $\mathbf{w}(n)$. If the filter coefficients change, the power in the error signal will also change. This is indicated by the changing height on the surface above w_0–w_1 the plane as the component values of $\mathbf{w}(n)$ are varied. Since the error surface is quadratic, a unique filter setting $\mathbf{w}(n) = \mathbf{w}^o$ will produce the minimum MSE, $\xi_{\min}$. In this two-weight case, the error surface is an elliptic paraboloid. If we cut the paraboloid with planes parallel to the w_0–w_1 plane, we obtain concentric ellipses of constant mean-square error. These ellipses are called the error contours of the error surface.

Example 8.7: Consider a second-order FIR filter with two coefficients w_0 and w_1. The reference signal $x(n)$ is a zero-mean white noise with unit variance. The desired signal is given as

$$d(n) = b_0x(n) + b_1x(n-1).$$

Plot the error surface and error contours.

From Equation (8.2.10), we obtain $\mathbf{R} = \begin{bmatrix} r_{xx}(0) & r_{xx}(1) \\ r_{xx}(1) & r_{xx}(0) \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$. From (8.2.8), we have $\mathbf{p} = \begin{bmatrix} r_{dx}(0) \\ r_{dx}(1) \end{bmatrix} = \begin{bmatrix} b_0 \\ b_1 \end{bmatrix}$. From (8.2.7), we get

$$\xi = E[d^2(n)] - 2\mathbf{p}^T\mathbf{w} + \mathbf{w}^T\mathbf{R}\mathbf{w} = (b_0^2 + b_1^2) - 2b_0w_0 - 2b_1w_1 + w_0^2 + w_1^2$$

Let $b_0 = 0.3$ and $b_1 = 0.5$, we have

$$\xi = 0.34 - 0.6w_0 - w_1 + w_0^2 + w_1^2.$$

The MATLAB script (`exam8_7a.m` in the software package) is used to plot the error surface shown in Figure 8.4(a) and the script `exam8_7b.m` is used to plot the error contours shown in Figure 8.4(b).

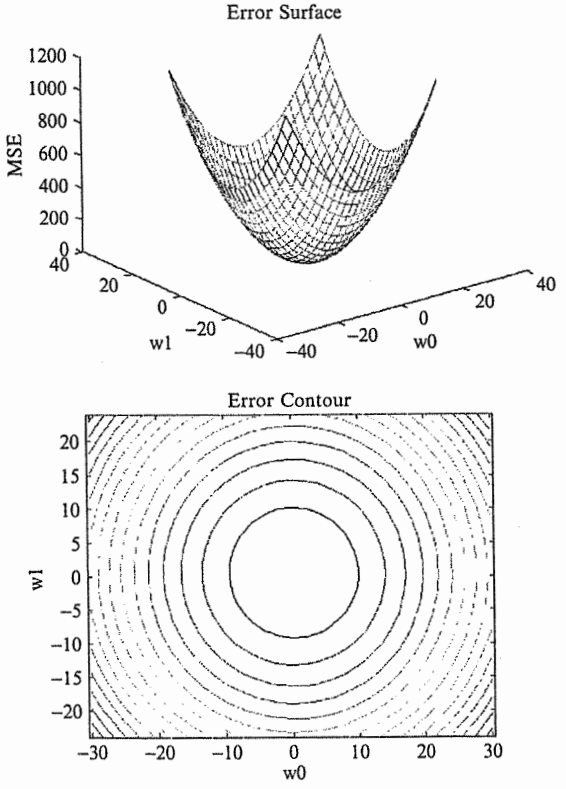


Figure 8.4 Performance surface and error contours, $L = 2$

One of the most important properties of the MSE surface is that it has only one global minimum point. At that minimum point, the tangents to the surface must be 0. Minimizing the MSE is the objective of many current adaptive methods such as the LMS algorithm.

8.2.3 Method of Steepest Descent

As shown in Figure 8.4, the MSE of (8.2.7) is a quadratic function of the weights that can be pictured as a positive-concave hyperparabolic surface. Adjusting the weights to minimize the error involves descending along this surface until reaching the 'bottom of the bowl.' Various gradient-based algorithms are available. These algorithms are based on making local estimates of the gradient and moving downward toward the bottom of the bowl. The selection of an algorithm is usually decided by the speed of convergence, steady-state performance, and the computational complexity.

The steepest-descent method reaches the minimum by following the direction in which the performance surface has the greatest rate of decrease. Specifically, an algorithm whose path follows the negative gradient of the performance surface. The steepest-descent method is an iterative (recursive) technique that starts from some initial (arbitrary) weight vector. It improves with the increased number of iterations. Geometrically, it is easy to see that with successive corrections of the weight vector in the direction of the steepest descent on the concave performance surface, we should arrive at its minimum, $\xi_{\min}$, at which point the weight vector components take on their optimum values. Let $\xi(0)$ represent the value of the MSE at time $n = 0$ with an arbitrary choice of the weight vector $\mathbf{w}(0)$. The steepest-descent technique enables us to descend to the bottom of the bowl, $\mathbf{w}^o$, in a systematic way. The idea is to move on the error surface in the direction of the tangent at that point. The weights of the filter are updated at each iteration in the direction of the negative gradient of the error surface.

The mathematical development of the method of steepest descent is easily seen from the viewpoint of a geometric approach using the MSE surface. Each selection of a filter weight vector $\mathbf{w}(n)$ corresponds to only one point on the MSE surface, $[\mathbf{w}(n), \xi(n)]$. Suppose that an initial filter setting $\mathbf{w}(0)$ on the MSE surface, $[\mathbf{w}(0), \xi(0)]$ is arbitrarily chosen. A specific orientation to the surface is then described using the directional derivatives of the surface at that point. These directional derivatives quantify the rate of change of the MSE surface with respect to the $\mathbf{w}(n)$ coordinate axes. The gradient of the error surface $\nabla\xi(n)$ is defined as the vector of these directional derivatives.

The concept of steepest descent can be implemented in the following algorithm:

$$\mathbf{w}(n+1) = \mathbf{w}(n) - \frac{\mu}{2}\nabla\xi(n) \tag{8.2.14}$$

where μ is a convergence factor (or step size) that controls stability and the rate of descent to the bottom of the bowl. The larger the value of μ, the faster the speed of descent. The vector $\nabla\xi(n)$ denotes the gradient of the error function with respect to $\mathbf{w}(n)$, and the negative sign increments the adaptive weight vector in the negative gradient direction. The successive corrections to the weight vector in the direction of

the steepest descent of the performance surface should eventually lead to the minimum mean-square error $\xi_{\min}$, at which point the weight vector reaches its optimum value $\mathbf{w}^o$.

When $\mathbf{w}(n)$ has converged to $\mathbf{w}^o$, that is, when it reaches the minimum point of the performance surface, the gradient $\nabla\xi(n) = \mathbf{0}$. At this time, the adaptation in (8.2.14) is stopped and the weight vector stays at its optimum solution. The convergence can be viewed as a ball placed on the 'bowl-shaped' MSE surface at the point $[\mathbf{w}(0), \xi(0)]$. If the ball was released, it would roll toward the minimum of the surface, and would initially roll in a direction opposite to the direction of the gradient, which can be interpreted as rolling towards the bottom of the bowl.

8.2.4 The LMS Algorithm

From (8.2.14), we see that the increment from $\mathbf{w}(n)$ to $\mathbf{w}(n+1)$ is in the negative gradient direction, so the weight tracking will closely follow the steepest descent path on the performance surface. However, in many practical applications the statistics of $d(n)$ and $x(n)$ are unknown. Therefore the method of steepest descent cannot be used directly, since it assumes exact knowledge of the gradient vector at each iteration. Widrow [13] used the instantaneous squared error, $e^2(n)$, to estimate the MSE. That is,

$$\hat{\xi}(n) = e^2(n). \tag{8.2.15}$$

Therefore the gradient estimate used by the LMS algorithm is

$$\nabla\hat{\xi}(n) = 2[\nabla e(n)]e(n). \tag{8.2.16}$$

Since $e(n) = d(n) - \mathbf{w}^T(n)\mathbf{x}(x)$, $\nabla e(n) = -\mathbf{x}(n)$, the gradient estimate becomes

$$\nabla\hat{\xi}(n) = -2\mathbf{x}(n)e(n). \tag{8.2.17}$$

Substituting this gradient estimate into the steepest-descent algorithm of (8.2.14), we have

$$\mathbf{w}(n+1) = \mathbf{w}(n) + \mu\mathbf{x}(n)e(n). \tag{8.2.18}$$

This is the well-known LMS algorithm, or stochastic gradient algorithm. This algorithm is simple and does not require squaring, averaging, or differentiating. The LMS algorithm provides an alternative method for determining the optimum filter coefficients without explicitly computing the matrix inversion suggested in (8.2.12).

Widrow's LMS algorithm is illustrated in Figure 8.5 and is summarized as follows:

1. Determine L, μ, and $\mathbf{w}(0)$, where L is the order of the filter, μ is the step size, and $\mathbf{w}(0)$ is the initial weight vector at time $n = 0$.

2. Compute the adaptive filter output

$$y(n) = \sum_{l=0}^{L-1} w_l(n)x(n-l). \tag{8.2.19}$$

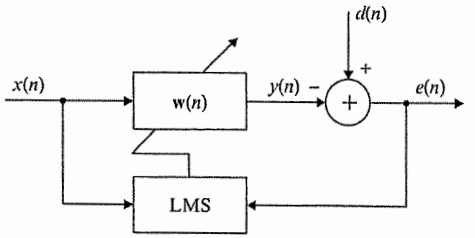


Figure 8.5 Block diagram of an adaptive filter with the LMS algorithm

3. Compute the error signal

$$e(n) = d(n) - y(n). \tag{8.2.20}$$

4. Update the adaptive weight vector from $\mathbf{w}(n)$ to $\mathbf{w}(n+1)$ by using the LMS algorithm

$$w_l(n+1) = w_l(n) + \mu x(n-l)e(n), \quad l = 0, 1, \ldots, L-1. \tag{8.2.21}$$

8.3 Performance Analysis

A detailed discussion of the performance of the LMS algorithm is available in many textbooks. In this section, we present some important properties of the LMS algorithm such as stability, convergence rate, and the excess mean-square error due to gradient estimation error.

8.3.1 Stability Constraint

As shown in Figure 8.5, the LMS algorithm involves the presence of feedback. Thus the algorithm is subject to the possibility of becoming unstable. From (8.2.18), we observe that the parameter μ controls the size of the incremental correction applied to the weight vector as we adapt from one iteration to the next. The mean weight convergence of the LMS algorithm from initial condition $\mathbf{w}(0)$ to the optimum filter $\mathbf{w}^o$ must satisfy

$$0 < \mu < \frac{2}{\lambda_{\max}}, \tag{8.3.1}$$

where $\lambda_{\max}$ is the largest eigenvalue of the autocorrelation matrix $\mathbf{R}$ defined in (8.2.10). Applying the stability constraint on μ given in (8.3.1) is difficult because of the computation of $\lambda_{\max}$ when L is large.

In practical applications, it is desirable to estimate $\lambda_{\max}$ using a simple method. From (8.2.10), we have

$$\text{tr}[\mathbf{R}] = Lr_{xx}(0) = \sum_{l=0}^{L-1} \lambda_l, \tag{8.3.2}$$

where tr[**R**] denotes the trace of matrix **R**. It follows that

$$\lambda_{\max} \leq \sum_{l=0}^{L-1} \lambda_l = Lr_{xx}(0) = LP_x, \tag{8.3.3}$$

where

$$P_x \equiv r_{xx}(0) = E[x^2(n)] \tag{8.3.4}$$

denotes the power of $x(n)$. Therefore setting

$$0 < \mu < \frac{2}{LP_x} \tag{8.3.5}$$

assures that (8.3.1) is satisfied.

Equation (8.3.5) provides some important information on how to select μ, and they are summarized as follows:

1. Since the upper bound on μ is inversely proportional to L, a small μ is used for large-order filters.

2. Since μ is made inversely proportional to the input signal power, weaker signals use a larger μ and stronger signals use a smaller μ. One useful approach is to normalize μ with respect to the input signal power P_x. The resulting algorithm is called the normalized LMS algorithm, which will be discussed in Section 8.4.

8.3.2 Convergence Speed

In the previous section, we saw that $\mathbf{w}(n)$ converges to $\mathbf{w}^o$ if the selection of μ satisfies (8.3.1). Convergence of the weight vector $\mathbf{w}(n)$ from $\mathbf{w}(0)$ to $\mathbf{w}^o$ corresponds to the convergence of the MSE from $\xi(0)$ to $\xi_{\min}$. Therefore convergence of the MSE toward its minimum value is a commonly used performance measurement in adaptive systems because of its simplicity. During adaptation, the squared error $e^2(n)$ is non-stationary as the weight vector $\mathbf{w}(n)$ adapts toward $\mathbf{w}^o$. The corresponding MSE can thus be defined only based on ensemble averages. A plot of the MSE versus time n is referred to as the learning curve for a given adaptive algorithm. Since the MSE is the performance criterion of LMS algorithms, the learning curve is a natural way to describe the transient behavior.

Each adaptive mode has its own time constant, which is determined by the overall adaptation constant μ and the eigenvalue λ_l associated with that mode. Overall convergence is clearly limited by the slowest mode. Thus the overall MSE time constant can be approximated as

$$\tau_{\text{mse}} \cong \frac{1}{\mu\lambda_{\min}}, \tag{8.3.6}$$

where $\lambda_{\min}$ is the minimum eigenvalue of the **R** matrix. Because τ_{mse} is inversely proportional to μ, we have a large τ_{mse} when μ is small (i.e., the speed of convergence is slow). If we use a large value of μ, the time constant is small, which implies faster convergence.

The maximum time constant $\tau_{\text{mse}} = 1/\mu\lambda_{\min}$ is a conservative estimate of filter performance, since only large eigenvalues will exert significant influence on the convergence time. Since some of the projections may be negligibly small, the adaptive filter error convergence may be controlled by fewer modes than the number of adaptive filter weights. Consequently, the MSE often converges more rapidly than the upper bound of (8.3.6) would suggest.

Because the upper bound of τ_{mse} is inversely proportional to $\lambda_{\min}$, a small $\lambda_{\min}$ can result in a large time constant (i.e., a slow convergence rate). Unfortunately, if $\lambda_{\max}$ is also very large, the selection of μ will be limited by (8.3.1) such that only a small μ can satisfy the stability constraint. Therefore if $\lambda_{\max}$ is very large and $\lambda_{\min}$ is very small, from (8.3.6), the time constant can be very large, resulting in very slow convergence. As previously noted, the fastest convergence of the dominant mode occurs for $\mu = 1/\lambda_{\max}$. Substituting this smallest step size into (8.3.6) results in

$$\tau_{\text{mse}} \leq \frac{\lambda_{\max}}{\lambda_{\min}}. \tag{8.3.7}$$

For stationary input and sufficiently small μ, the speed of convergence of the algorithm is dependent on the eigenvalue spread (the ratio of the maximum to minimum eigenvalues) of the matrix **R**.

As mentioned in the previous section, the eigenvalues $\lambda_{\max}$ and $\lambda_{\min}$ are very difficult to compute. However, there is an efficient way to estimate the eigenvalue spread from the spectral dynamic range. That is,

$$\frac{\lambda_{\max}}{\lambda_{\min}} \leq \frac{\max |X(\omega)|^2}{\min |X(\omega)|^2}, \tag{8.3.8}$$

where $X(\omega)$ is DTFT of $x(n)$ and the maximum and minimum are calculated over the frequency range $0 \leq \omega \leq \pi$. From (8.3.7) and (8.3.8), input signals with a flat (white) spectrum have the fastest convergence speed.

8.3.3 Excess Mean-Square Error

The steepest-descent algorithm in (8.2.14) requires knowledge of the gradient $\nabla\xi(n)$, which must be estimated at each iteration. The estimated gradient $\nabla\hat{\xi}(n)$ produces the gradient estimation noise. After the algorithm converges, i.e., $\mathbf{w}(n)$ is close to $\mathbf{w}^o$, the true gradient $\nabla\xi(n) \approx \mathbf{0}$. However, the gradient estimator $\nabla\hat{\xi}(n) \neq \mathbf{0}$. As indicated by the update of Equation (8.2.14), perturbing the gradient will cause the weight vector $\mathbf{w}(n+1)$ to move away from the optimum solution $\mathbf{w}^o$. Thus the gradient estimation

noise prevents $\mathbf{w}(n+1)$ from staying at $\mathbf{w}^o$ in steady state. The result is that $\mathbf{w}(n)$ varies randomly about $\mathbf{w}^o$. Because $\mathbf{w}^o$ corresponds to the minimum MSE, when $\mathbf{w}(n)$ moves away from $\mathbf{w}^o$, it causes $\xi(n)$ to be larger than its minimum value, $\xi_{\min}$, thus producing excess noise at the filter output.

The excess MSE, which is caused by random noise in the weight vector after convergence, is defined as the average increase of the MSE. For the LMS algorithm, it can be approximated as

$$\xi_{\text{excess}} \approx \frac{\mu}{2} L P_x \xi_{\min}. \tag{8.3.9}$$

This approximation shows that the excess MSE is directly proportional to μ. The larger the value of μ, the worse the steady-state performance after convergence. However, Equation (8.3.6) shows that a larger μ results in faster convergence. There is a design trade-off between the excess MSE and the speed of convergence.

The optimal step size μ is difficult to determine. Improper selection of μ might make the convergence speed unnecessarily slow or introduce excess MSE. If the signal is non-stationary and real-time tracking capability is crucial for a given application, then use a larger μ. If the signal is stationary and convergence speed is not important, use a smaller μ to achieve better performance in a steady state. In some practical applications, we can use a larger μ at the beginning of the operation for faster convergence, then use a smaller μ to achieve better steady-state performance.

The excess MSE, ξ_{excess}, in (8.3.9) is also proportional to the filter order L, which means that a larger L results in larger algorithm noise. From (8.3.5), a larger L implies a smaller μ, resulting in slower convergence. On the other hand, a large L also implies better filter characteristics such as sharp cutoff. There exists an optimum order L for any given application. The selection of L and μ also will affect the finite-precision error, which will be discussed in Section 8.6.

In a stationary environment, the signal statistics are unknown but fixed. The LMS algorithm gradually learns the required input statistics. After convergence to a steady state, the filter weights jitter around the desired fixed values. The algorithm performance is determined by both the speed of convergence and the weight fluctuations in steady state. In the non-stationary case, the algorithm must continuously track the time-varying statistics of the input. Performance is more difficult to assess.

8.4 Modified LMS Algorithms

The LMS algorithm described in the previous section is the most widely used adaptive algorithm for practical applications. In this section, we present two modified algorithms that are the direct variants of the basic LMS algorithm.

8.4.1 Normalized LMS Algorithm

The stability, convergence speed, and fluctuation of the LMS algorithm are governed by the step size μ and the reference signal power. As shown in (8.3.5), the maximum stable

step-size μ is inversely proportional to the filter order L and the power of the reference signal $x(n)$. One important technique to optimize the speed of convergence while maintaining the desired steady-state performance, independent of the reference signal power, is known as the normalized LMS algorithm (NLMS). The NLMS algorithm is expressed as

$$\mathbf{w}(n+1) = \mathbf{w}(n) + \mu(n)\mathbf{x}(n)e(n), \tag{8.4.1}$$

where $\mu(n)$ is an adaptive step size that is computed as

$$\mu(n) = \frac{\alpha}{L\hat{P}_x(n),} \tag{8.4.2}$$

where $\hat{P}_x(n)$ is an estimate of the power of $x(n)$ at time n, and α is a normalized step size that satisfies the criterion

$$0 < \alpha < 2. \tag{8.4.3}$$

The commonly used method to estimate $\hat{P}_x(n)$ sample-by-sample is introduced in Section 3.2.1. Some useful implementation considerations are given as follows:

1. Choose $\hat{P}_x(0)$ as the best a priori estimate of the reference signal power.

2. Since it is not desirable that the power estimate $\hat{P}_x(n)$ be zero or very small, a software constraint is required to ensure that $\mu(n)$ is bounded even if $\hat{P}_x(n)$ is very small when the signal is absent for a long time. This can be achieved by modifying (8.4.2) as

$$\mu(n) = \frac{\alpha}{L\hat{P}_x(n) + c,} \tag{8.4.4}$$

where c is a small constant.

8.4.2 Leaky LMS Algorithm

Insufficient spectral excitation of the LMS algorithm may result in divergence of the adaptive weights. In that case, the solution is not unique and finite-precision effects can cause the unconstrained weights to grow without bound, resulting in overflow during the weight update process. This long-term instability is undesirable for real-time applications.

Divergence can often be avoided by means of a 'leaking' mechanism used during the weight update calculation. This is called the leaky LMS algorithm and is expressed as

$$\mathbf{w}(n+1) = \nu\mathbf{w}(n) + \mu\mathbf{x}(n)e(n), \tag{8.4.5}$$

where ν is the leakage factor with $0 < \nu \leq 1$. It can be shown that leakage is the deterministic equivalent of adding low-level white noise. Therefore this approach results

in some degradation in adaptive filter performance. The value of the leakage factor is determined by the designer on an experimental basis as a compromise between robustness and loss of performance of the adaptive filter. The leakage factor introduces a bias on the long-term coefficient estimation. The excess error power due to the leakage is proportional to $[(1-\nu)/\mu]^2$. Therefore $(1-\nu)$ should be kept smaller than μ in order to maintain an acceptable level of performance. For fixed-point hardware realization, multiplication of each coefficient by ν, as shown in (8.4.5), can lead to the introduction of roundoff noise, which adds to the excess MSE. Therefore the leakage effects must be incorporated into the design procedure for determining the required coefficient and internal data wordlength. The leaky LMS algorithm not only prevents unconstrained weight overflow, but also limits the output power in order to avoid nonlinear distortion.

8.5 Applications

The desirable features of an adaptive filter are the ability to operate in an unknown environment and to track time variations of the input signals, making it a powerful algorithm for DSP applications. The essential difference between various applications of adaptive filtering is where the signals $x(n)$, $d(n)$, $y(n)$, and $e(n)$ are connected. There are four basic classes of adaptive filtering applications: identification, inverse modeling, prediction, and interference canceling.

8.5.1 Adaptive System Identification

System identification is an experimental approach to the modeling of a process or a plant. The basic idea is to measure the signals produced by the system and to use them to construct a model. The paradigm of system identification is illustrated in Figure 8.6, where $P(z)$ is an unknown system to be identified and $W(z)$ is a digital filter used to model $P(z)$. By exciting both the unknown system $P(z)$ and the digital model $W(z)$ with the same excitation signal $x(n)$ and measuring the output signals $y(n)$ and $d(n)$, we can determine the characteristics of $P(z)$ by adjusting the digital model $W(z)$ to minimize the difference between these two outputs. The digital model $W(z)$ can be an FIR filter or an IIR filter.

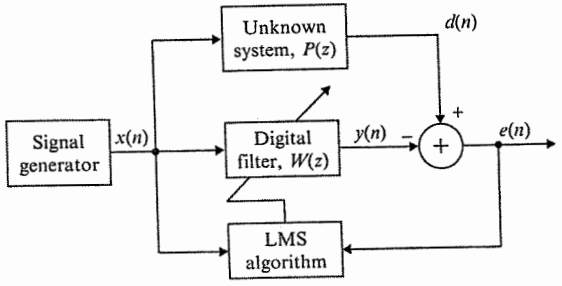


Figure 8.6 Block diagram of adaptive system identification using the LMS algorithm

Adaptive system identification is a technique that uses an adaptive filter for the model $W(z)$. This section presents the application of adaptive estimation techniques for direct system modeling. This technique has been widely applied in echo cancellation, which will be introduced in Sections 9.4 and 9.5. A further application for system modeling is to estimate various transfer functions in active noise control systems [8].

Adaptive system identification is a very important procedure that is used frequently in the fields of control systems, communications, and signal processing. The modeling of a single-input/single-output dynamic system (or plant) is shown in Figure 8.6, where $x(n)$, which is usually white noise, is applied simultaneously to the adaptive filter and the unknown system. The output of the unknown system then becomes the desired signal, $d(n)$, for the adaptive filter. If the input signal $x(n)$ provides sufficient spectral excitation, the adaptive filter output $y(n)$ will approximate $d(n)$ in an optimum sense after convergence.

Identification could mean that a set of data is collected from the system, and that a separate procedure is used to construct a model. Such a procedure is usually called off-line (or batch) identification. In many practical applications, however, the model is sometimes needed on-line during the operation of the system. That is, it is necessary to identify the model at the same time that the data set is collected. The model is updated at each time instant that a new data set becomes available. The updating is performed with a recursive adaptive algorithm such as the LMS algorithm.

As shown in Figure 8.6, it is desired to learn the structure of the unknown system from knowledge of its input $x(n)$ and output $d(n)$. If the unknown time-invariant system $P(z)$ can be modeled using an FIR filter of order L, the estimation error is given as

$$e(n) = d(n) - y(n) = \sum_{l=0}^{L-1} [p(l) - w_l(n)]x(n-l), \tag{8.5.1}$$

where $p(l)$ is the impulse response of the unknown plant.

By choosing each $w_l(n)$ close to each $p(l)$, the error will be made small. For white-noise input, the converse also holds: minimizing $e(n)$ will force the $w_l(n)$ to approach $p(l)$, thus identifying the system

$$w_l(n) \approx p(l), \quad l = 0, 1, \ldots, L-1. \tag{8.5.2}$$

The basic concept is that the adaptive filter adjusts itself, intending to cause its output to match that of the unknown system. When the difference between the physical system response $d(n)$ and adaptive model response $y(n)$ has been minimized, the adaptive model approximates $P(z)$. In actual applications, there will be additive noise present at the adaptive filter input and so the filter structure will not exactly match that of the unknown system. When the plant is time varying, the adaptive algorithm has the task of keeping the modeling error small by continually tracking time variations of the plant dynamics.

8.5.2 Adaptive Linear Prediction

Linear prediction is a classic signal processing technique that provides an estimate of the value of an input process at a future time where no measured data is yet available. The

techniques have been successfully applied to a wide range of applications such as speech coding and separating signals from noise. As illustrated in Figure 8.7, the time-domain predictor consists of a linear prediction filter in which the coefficients $w_l(n)$ are updated with the LMS algorithm. The predictor output $y(n)$ is expressed as

$$y(n) = \sum_{l=0}^{L-1} w_l(n)x(n - \Delta - l), \tag{8.5.3}$$

where the delay Δ is the number of samples involved in the prediction distance of the filter. The coefficients are updated as

$$\mathbf{w}(n+1) = \mathbf{w}(n) + \mu\mathbf{x}(n - \Delta)e(n), \tag{8.5.4}$$

where $\mathbf{x}(n-\Delta) = [x(n-\Delta)\ x(n-\Delta-1)\ \ldots\ x(n-\Delta-L+1)]^T$ is the delayed reference signal vector, and $e(n) = x(n) - y(n)$ is the prediction error. Proper selection of the prediction delay Δ allows improved frequency estimation performance for multiple sinusoids in white noise.

Now consider the adaptive predictor for enhancing an input of M sinusoids embedded in white noise, which is of the form

$$x(n) = s(n) + v(n) = \sum_{m=0}^{M-1} A_m \sin(\omega_m n + \phi_m) + v(n), \tag{8.5.5}$$

where $v(n)$ is white noise with uniform noise power σ_v^2. In this application, the structure shown in Figure 8.7 is called the adaptive line enhancer, which provides an efficient means for the adaptive tracking of the sinusoidal components of a received signal $x(n)$ and separates these narrowband signals $s(n)$ from broadband noise $v(n)$. This technique has been shown effective in practical applications when there is insufficient a priori knowledge of the signal and noise parameters.

As shown in Figure 8.7, we want the highly correlated components of $x(n)$ to appear in $y(n)$. This is accomplished by adjusting the weights to minimize the expected mean-square value of the error signal $e(n)$. This causes an adaptive filter $W(z)$ to form

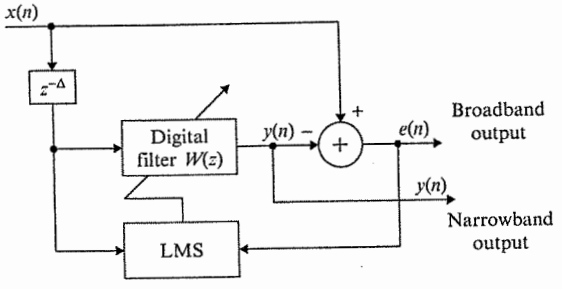


Figure 8.7 Block diagram of an adaptive predictor

narrowband bandpass filters centered at the frequency of the sinusoidal components. The noise component of the input is rejected, while the phase difference (caused by Δ) of the narrowband signals is readjusted so that they can cancel correlated components in $d(n)$ to minimize the error signal $e(n)$. In this case, the output $y(n)$ is the enhanced signal, which contains multiple sinusoidals as expressed in (8.5.5).

In many digital communications and signal detection applications, the desired broadband (spread-spectrum) signal $v(n)$ is corrupted by an additive narrowband interference $s(n)$. From a filtering viewpoint, the objective of an adaptive filter is to form a notch in the frequency band occupied by the interference, thus suppressing the narrowband noise. The narrowband characteristics of the interference allow $W(z)$ to estimate $s(n)$ from past samples of $x(n)$ and to subtract the estimate from $x(n)$. The error signal $e(n)$ in Figure 8.7 consists of desired broadband signals. In this application, the desired output from the overall interference suppression filter is $e(n)$.

8.5.3 Adaptive Noise Cancellation

The wide spread use of cellular phones has significantly increased the use of communication systems in high noise environments. Intense background noise, however, often corrupts speech and degrading the performance of many communication systems. Existing signal processing techniques such as speech coding, automatic speech recognition, speaker identification, channel transmission, and echo cancellation are developed under noise-free assumptions. These techniques could be employed in noisy environments if a front-end noise suppression algorithm sufficiently reduces additive noise. Noise reduction is becoming increasingly important with the development and application of hands-free and voice-activated cellular phones.

Single-channel noise reduction methods involve Wiener filtering, Kalman filtering, and spectral subtraction. In the dual-channel systems, a second sensor provides a reference noise to better characterize changing noise statistics, which is necessary for dealing with non-stationary noise. The most widely used dual-channel adaptive noise canceler (ANC) employs an adaptive filter with the LMS algorithm to cancel the noise component in the primary signal picked up by the primary sensor.

As illustrated in Figure 8.8, the basic concept of adaptive noise cancellation is to process signals from two sensors and to reduce the level of the undesired noise with adaptive filtering techniques. The primary sensor is placed close to the signal source in order to pick up the desired signal. However, the primary sensor output also contains noise from the noise source. The reference sensor is placed close to the noise source to sense only the noise. This structure takes advantage of the correlation between the noise signals picked up by the primary sensor and those picked up by the reference sensor.

A block diagram of the adaptive noise cancellation system is illustrated in Figure 8.9, where $P(z)$ represents the transfer function between the noise source and the primary sensor. The canceler has two inputs: the primary input $d(n)$ and the reference input $x(n)$. The primary input $d(n)$ consists of signal $s(n)$ plus noise $x'(n)$, i.e., $d(n) = s(n) + x'(n)$, which is highly correlated with $x(n)$ since they are derived from the same noise source. The reference input simply consists of noise $x(n)$. The objective of the adaptive filter is to use the reference input $x(n)$ to estimate the noise $x'(n)$. The filter output $y(n)$, which is an

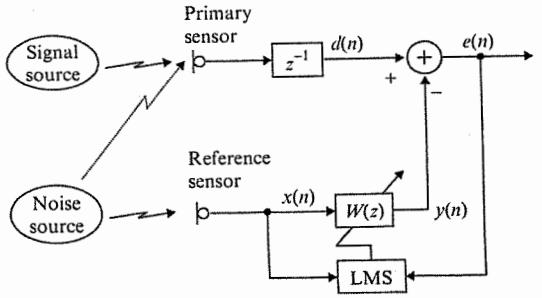


Figure 8.8 Basic concept of adaptive noise canceling

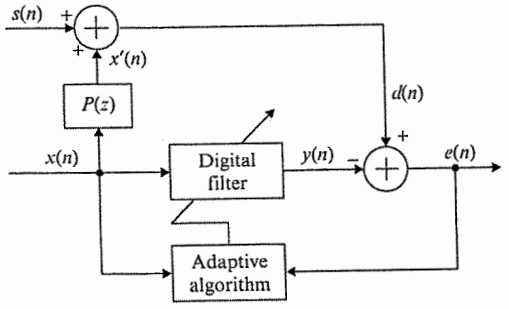


Figure 8.9 Block diagram of adaptive noise canceler

estimate of noise $x'(n)$, is then subtracted from the primary channel signal $d(n)$, producing $e(n)$ as the desired signal plus reduced noise.

To minimize the residual error $e(n)$, the adaptive filter $W(z)$ will generate an output $y(n)$ that is an approximation of $x'(n)$. Therefore the adaptive filter $W(z)$ will converge to the unknown plant $P(z)$. This is the adaptive system identification scheme discussed in Section 8.5.1. To apply the ANC effectively, the reference noise picked up by the reference sensor must be highly correlated with the noise components in the primary signal. This condition requires a close spacing between the primary and reference sensors. Unfortunately, it is also critical to avoid the signal components from the signal source being picked up by the reference sensor. This 'crosstalk' effect will degrade the performance of ANC because the presence of the signal components in reference signal will cause the ANC to cancel the desired signal along with the undesired noise. The performance degradation of ANC with crosstalk includes less noise reduction, slower convergence, and reverberant distortion in the desired signal.

Crosstalk problems may be eliminated by placing the primary sensor far away from the reference sensor. Unfortunately, this arrangement requires a large-order filter in order to obtain adequate noise reduction. For example, a separation of a few meters between the two sensors requires a filter with 1500 taps to achieve 20 dB noise reduction. The long filter increases excess mean-square error and decreases the tracking ability of

ANC because the step size must be reduced to ensure stability. Furthermore, it is not always feasible to place the reference sensor far away from the signal source. The second method for reducing crosstalk is to place an acoustic barrier (an oxygen mask in an aircraft cockpit, for example) between the primary and reference sensors. However, many applications do not allow an acoustic barrier between sensors, and a barrier may reduce the correlation of the noise component in the primary and reference signals. The third technique involves allowing the adaptive algorithm to update filter coefficients during silent intervals in the speech. Unfortunately, this method depends on a reliable speech detector that is very application dependent. This technique also fails to track the environment changes during the speech periods.

8.5.4 Adaptive Notch Filters

In certain situations, the primary input is a broadband signal with an undesired narrowband (sinusoidal) interference. The conventional method of eliminating such sinusoidal interference is by using a notch filter tuned to the frequency of the interference. To design the filter, we need to estimate the precise frequency of the interference. A very narrow notch is usually desired in order to filter out the interference without seriously distorting the signal of interest. The advantages of the adaptive notch filter are that it offers an infinite null, and the capability to adaptively track the frequency of the interference. The adaptive notch filter is especially useful when the interfering sinusoid drifts slowly in frequency. In this section, we will present two adaptive notch filters.

The adaptive structure shown in Figure 8.7 can be applied to enhance the broadband signal, which is corrupted by multiple narrowband components. For example, the input signal expressed in (8.5.5) consists of a broadband signal $v(n)$, which is music. In this application, $e(n)$ is the desired output that consists of enhanced broadband music signals since the narrowband components are readjusted by $W(z)$ so that they can cancel correlated components in $d(n)$. The adaptive system between the input $x(n)$ and the output $e(n)$ is an adaptive notch filter, which forms several notch filters centered at the frequency of the sinusoidal components.

A sinusoid can be used as a reference signal for canceling each component of narrowband noise. When a sinewave is employed as the reference input, the LMS algorithm becomes an adaptive notch filter, which removes the primary spectral components within a narrowband centered about the reference frequency. Furthermore, multiple harmonic disturbances can be handled if the reference signal is composed of a number of sinusoids.

A single-frequency adaptive notch filter with two adaptive weights is illustrated in Figure 8.10. The reference input is a cosine signal

$$x(n) = x_0(n) = A\cos(\omega_0 n). \tag{8.5.6}$$

A 90° phase shifter is used to produce the quadrature reference signal

$$x_1(n) = A\sin(\omega_0 n). \tag{8.5.7}$$

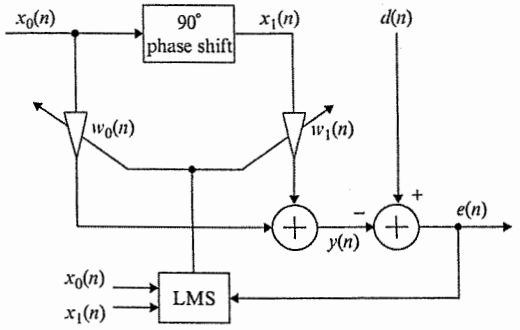


Figure 8.10 Single-frequency adaptive notch filter

Digital Hilbert transform filters can be employed for this purpose. Instead of using cosine generator and a phase shifter, the recursive quadrature oscillator given in Figure 6.23 can be used to generate both sine and cosine signals simultaneously. For a reference sinusoidal signal, two filter coefficients are needed.

The LMS algorithm employed in Figure 8.10 is summarized as follows:

$$y(n) = w_0(n)x_0(n) + w_1(n)x_1(n), \tag{8.5.8}$$

$$e(n) = d(n) - y(n), \tag{8.5.9}$$

$$w_l(n+1) = w_l(n) + \mu x_l(n)e(n), \quad l = 0, 1. \tag{8.5.10}$$

Note that the two-weight adaptive filter $W(z)$ shown in Figure 8.10 can be replaced with a general L-weight adaptive FIR filter for a multiple sinusoid reference input $x(n)$. The reference input supplies a correlated version of the sinusoidal interference that is used to estimate the composite sinusoidal interfering signal contained in the primary input $d(n)$.

The single-frequency adaptive notch filter has the property of a tunable notch filter. The center frequency of the notch filter depends on the sinusoidal reference signal, whose frequency is equal to the frequency of the primary sinusoidal noise. Therefore the noise at that frequency is attenuated. This adaptive notch filter provides a simple method for tracking and eliminating sinusoidal interference.

Example 8.8: For a stationary input and sufficiently small μ, the convergence speed of the LMS algorithm is dependent on the eigenvalue spread of the input autocorrelation matrix. For $L = 2$ and the reference input is given in (8.5.6), the autocorrelation matrix can be expressed as

$$\mathbf{R} = E\begin{bmatrix} x_0(n)x_0(n) & x_0(n)x_1(n) \\ x_1(n)x_0(n) & x_1(n)x_1(n) \end{bmatrix}$$

$$= E\begin{bmatrix} A^2\cos^2(\omega_0 n) & A^2\cos(\omega_0 n)\sin(\omega_0 n) \\ A^2\sin(\omega_0 n)\cos(\omega_0 n) & A^2\sin^2(\omega_0 n) \end{bmatrix} = \begin{bmatrix} \frac{A^2}{2} & 0 \\ 0 & \frac{A^2}{2} \end{bmatrix}.$$

This equation shows that because of the 90° phase shift, $x_0(n)$ is orthogonal to $x_1(n)$ and the off-diagonal terms in the $\mathbf{R}$ matrix are 0. The eigenvalues λ_1 and λ_2 of the $\mathbf{R}$ matrix are identical and equal to $A^2/2$. Therefore the system has very fast convergence since the eigenvalue spread equals 1. The time constant of the adaptation is approximated as

$$\tau_{\text{mse}} \leq \frac{1}{\mu\lambda} = \frac{2}{\mu A^2},$$

which is determined by the power of the reference sinewave and the step size μ.

8.5.5 Adaptive Channel Equalization

In digital communications, considerable effort has been devoted to the development of data-transmission systems that utilize the available telephone channel bandwidth efficiently. The transmission of high-speed data through a channel is limited by intersymbol interference (ISI) caused by distortion in the transmission channel. High-speed data transmission through channels with severe distortion can be achieved in several ways, such as (1) by designing the transmit and receive filters so that the combination of filters and channel results in an acceptable error from the combination of ISI and noise; and (2) by designing an equalizer in the receiver that counteracts the channel distortion. The second method is the most commonly used.

As illustrated in Figure 8.11, the received signal $y(n)$ is different from the original signal $x(n)$ because it was distorted by the overall channel transfer function $C(z)$, which includes the transmit filter, the transmission medium, and the receive filter. To recover the original signal, $x(n)$, we need to process $y(n)$ using the equalizer $W(z)$, which is the inverse of the channel's transfer function $C(z)$ to compensate for the channel distortion. That is, we have to design the equalizer

$$W(z) = \frac{1}{C(z)}, \tag{8.5.11}$$

i.e., $C(z)W(z) = 1$ such that $\hat{x}(n) = x(n)$.

In practice, the telephone channel is time varying and is unknown in the design stage due to variations in the transmission medium. The transmit and receive filters that are

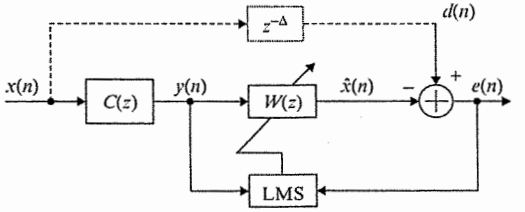


Figure 8.11 Cascade of channel with an ideal adaptive channel equalizer

designed based on the average channel characteristics may not adequately reduce intersymbol interference. Thus we need an adaptive equalizer that provides precise compensation over the time-varying channel. As illustrated in Figure 8.11, the adaptive channel equalizer is an adaptive filter with coefficients that are adjusted using the LMS algorithm.

As shown in Figure 8.11, an adaptive filter requires the desired signal $d(n)$ for computing the error signal $e(n)$ for the LMS algorithm. In theory, the delayed version of the transmitted signal, $x(n-\Delta)$, is the desired response for the adaptive equalizer $W(z)$. However, with the adaptive filter located in the receiver, the desired signal generated by the transmitter is not available at the receiver. The desired signal may be generated locally in the receiver using two methods. A decision-directed algorithm, in which the equalized signal $\hat{x}(n)$ is sliced to form the desired signal, is the simplest and can be used for channels that have only a moderate amount of distortion. However, if the error rate of the data derived by slicing is too high, the convergence may be seriously impaired. In this case, the training of an equalizer by using that sequence is agreed on beforehand by the transmitter and the receiver.

During the training stage, the adaptive equalizer coefficients are adjusted by transmitting a short training sequence. This known transmitted sequence is also generated in the receiver and is used as the desired signal $d(n)$ for the LMS algorithm. A widely used training signal consists of pseudo-random noise (will be introduced in Section 9.2) with a broad and flat power spectrum. After the short training period, the transmitter begins to transmit the data sequence. In order to track the possible slow time variations in the channel, the equalizer coefficients must continue to be adjusted while receiving data. In this data mode, the output of the equalizer, $\hat{x}(n)$, is used by a decision device (slicer) to produce binary data. Assuming the output of the decision device is correct, the binary sequence can be used as the desired signal $d(n)$ to generate the error signal for the LMS algorithm.

An equalizer for a one-dimensional baseband system has real input signals and filter coefficients. However, for a two-dimensional quadrature amplitude modulation (QAM) system, both signals and coefficients are complex. All operations must use complex arithmetic and the complex LMS algorithm expressed as

$$\mathbf{w}(n+1) = \mathbf{w}(n) + \mu \mathbf{x}^*(n) e(n), \tag{8.5.12}$$

where * denotes complex conjugate and

$$\mathbf{x}(n) = \mathbf{x}_R(n) + j\mathbf{x}_I(n) \tag{8.5.13}$$

and

$$\mathbf{w}(n) = \mathbf{w}_R(n) + j\mathbf{w}_I(n). \tag{8.5.14}$$

In (8.5.13) and (8.5.14), the subscript R and I represent real and imaginary parts of complex numbers. The complex output $\hat{x}(n)$ is given by

$$\hat{x}(n) = \mathbf{w}^T(n)\mathbf{x}(n). \tag{8.5.15}$$

Since all multiplications are complex, the equalizer usually requires four times as many multiplications.

8.6 Implementation Considerations

The adaptive algorithms introduced in previous sections assume the use of infinite precision for the signal samples and filter coefficients. In practice, an adaptive algorithm is implemented on finite-precision hardware. It is important to understand the finite-wordlength effects of adaptive algorithms in meeting design specifications.

8.6.1 Computational Issues

The coefficient update defined in (8.2.21) requires $L+1$ multiplications and L additions by multiplying $\mu^* e(n)$ outside the loop. Given the input vector $\mathbf{x}(n)$ stored in the array `x[]`, the error signal `en`, the weight vector `w[]`, the step size `mu`, and the filter order `L`, Equation (8.2.21) can be implemented in C as follows:

```
uen = mu*en;              /* u*e(n)              */
for(l = 0, l < L, l++)    /* l = 0, 1, ..., L-1  */
{
   w[l] += uen*x[l];      /* LMS update          */
}
```

The architecture of most commercially available DSP chips has been optimized for convolution operations to compute output $y(n)$ given in (8.2.19) in L instruction cycles. However, the weight update operations in (8.2.21) cannot take advantage of this special architecture because each update cycle involves loading the weight value into the accumulator, performing a multiply–add operation, and storing the result back into memory. With typical DSP architecture, the memory transfer operations take as many machine cycles as the multiply–add, thus resulting in a dominant computational burden involved in the weight update.

Let T_p denote the total processing time for each input sample. Real-time processing requires that $T_p < T$, where T is the sampling period. This means that if the algorithm is too complicated, we need to reduce the complexity of computation to allow $T_p < T$. Because the output convolution is more important and can be implemented very efficiently using DSP chips, skipping part of the weight update is suggested. That is, the simplest solution is to update only a fraction of the coefficients each sample period. The principal cost is a somewhat slower convergence rate.

In a worst-case scenario, we might update only one weight during a sample period. The next weight would be updated for the next sample period, and so on. When the computing power permits, a group of samples can be updated during each sample period.